

Introduction to Data Centre Management Using a Simulation Game

First Author¹[0000–1111–2222–3333], Second Author^{2,3}[1111–2222–3333–4444], and
Third Author³[2222–3333–4444–5555]

¹ Princeton University, Princeton NJ 08544, USA

² Springer Heidelberg, Tiergartenstr. 17, 69121 Heidelberg, Germany

lncs@springer.com

<http://www.springer.com/gp/computer-science/lncs>

³ ABC Institute, Rupert-Karls-University Heidelberg, Heidelberg, Germany
{abc,lncs}@uni-heidelberg.de

Abstract. The abstract should briefly summarize the contents of the paper in 150–250 words.

Keywords: Data centers simulation · Management in data centers · Simulation games · Game-Based Learning.

1 Introduction

The video game industry has experienced substantial growth over the past decade. In many countries, its economic impact has already surpassed that of traditional media sectors such as film, television, and newspapers. As the industry continues to expand in both scale and relevance, the application of video games increasingly extends beyond entertainment.

Serious games and simulation-based games provide players with interactive experiences that incorporate educational content. However, studies such as *Game-Based Learning with Computers: Learning, Simulations* [8] demonstrate that even games developed exclusively for entertainment require players to acquire new knowledge, develop complex skills, and devise effective strategies in order to progress through levels and overcome challenges.

Similarly, research such as *Video Games and Indirect Learning* by Monica Miller [7] suggests that the content of video games can also facilitate indirect learning, that is, learning that occurs without the game being explicitly designed for educational purposes.

Motivated by these findings, this work aims to investigate whether players acquire knowledge while interacting with **Port 0**, a resource management video game in which the player is responsible for operating a small data center within a fictional setting. Throughout the game, players must purchase, configure, and manage servers, fulfil client requests, and defend the infrastructure against cyberattacks.

Although the game is set in a science fiction universe, it incorporates several simulation-based mechanics, including word-based mini-games and server configuration tasks. The objective is to provide an engaging and enjoyable gameplay experience while simultaneously promoting knowledge acquisition, both through direct learning mechanisms embedded in the mini-games and through indirect learning arising from the contextual environment in which the player operates.

2 Related Work

The number of video games that convey knowledge to players, either directly or indirectly, is considerably large, regardless of whether they fall within the category of serious games. Alongside the video game industry, several authors have conducted studies and research on learning through video games in recent years. Trends related to the technology field, such as **data center** and **technology**, have also become increasingly popular in recent years.

Therefore, throughout this section, the objective is to analyze the learning strategies applied to video games and understand how they produce learning outcomes. Finally, the growth of trends related to the context of the game (**Port 0**) is analyzed.

2.1 Learning through observation

Learning through observation, also referred to as observational learning, is the process by which beginners absorb rules, initial steps, and complex procedures simply by watching experienced players or interacting with simulations and/or tutorials [8]. When integrated into video games or simulation systems, this type of learning acts as a pedagogical "scaffold" for the learner [8].

Several titles, such as CodeCombat, Manufactoria, and Labor Support Game, employ this learning strategy.

In CodeCombat, for example, the developers implemented a split-screen interface where the code is highlighted line by line while the hero performs the corresponding actions on screen, allowing players to directly observe the outcome of each command [9].

A similar approach to CodeCombat was adopted to convey the meaning of commands within the WordRush minigame. This minigame is discussed in Subsection ??.

2.2 Self-tutorage and Trial and Error

Learning through trial and error is an active and exploratory learning strategy in which individuals test hypotheses in real time to determine which actions lead to successful outcomes and which result in failure [9].

Trial and error is also a process in which learners repeatedly attempt missions or tasks until they achieve success or the highest possible score, allowing them to

understand complex concepts through direct experience with their own mistakes [10,11].

Feedback loops can be used as a mechanism to enhance this technique, as they create a repetitive cycle in which the system responds to the player's actions, thereby providing continuous learning opportunities [12].

Accordingly, the developed project, **Port 0**, incorporates gameplay loops in which the player is required to repeat a set of tasks. This aspect is discussed in greater detail in Section 3.

Finally, self-tutorage is a common practice, particularly in fields such as programming, involving the development of solutions through continuous testing of what works and what does not in real time. This process functions as a form of self-learning and continuous skill development [13].

2.3 Indirect learning

Indirect learning is a process in which knowledge is acquired as a by-product of another activity, such as leisure [14,13]. This process occurs implicitly, unconsciously, or naturally, often without the individual initially realizing that learning is taking place [14].

This learning mechanism is effective because, rather than explicitly teaching content, games create environments in which knowledge is constructed through experience, problem-solving, and interaction with complex systems [7].

Commercial games such as *No Man's Sky* [15] and *The Legend of Zelda: Breath of the Wild* [16] nevertheless promote knowledge and skills such as:

- Experimentation;
- Scientific thinking;
- Creativity;
- Complex problem-solving;
- Resource management.

According to the study conducted by Monica Miller, *No Man's Sky* [15] fosters scientific curiosity not because it explicitly teaches science, but because its gameplay structure encourages players to adopt behaviors similar to those used in scientific inquiry [7].

Similarly, **Port 0** seeks to promote scientific curiosity through its environment, visual elements, and gameplay tasks.

2.4 Hype surrounding the technology field

Hype is defined as "a situation in which something is advertised and discussed in newspapers, on television, etc. a lot in order to attract everyone's interest" [17].

To evaluate the hype surrounding topics such as *technology* and *data center*, the *Google Trends* tool was used. Google Trends measures the relative search

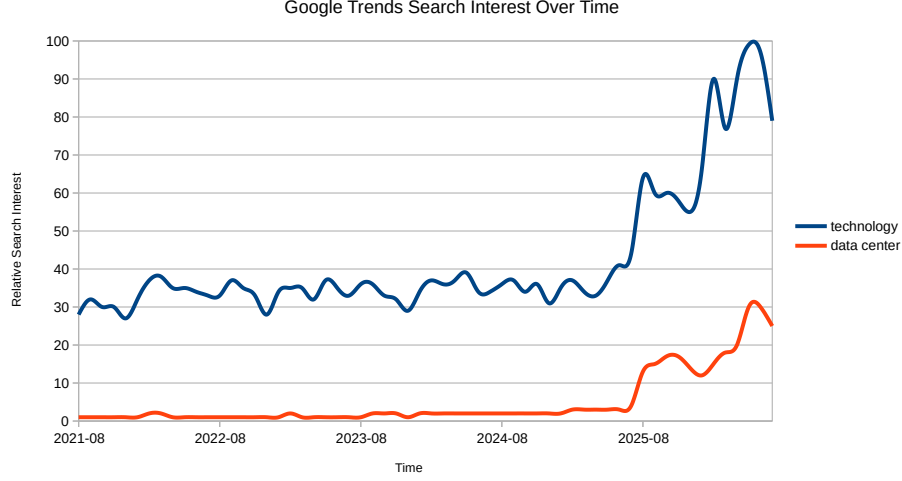


Fig. 1. Relative Search Interest for *Data Center* and *Technology*.

interest for terms entered by users into Google’s search engine over time and across different geographical regions.

The data obtained from *Google Trends* are presented in Figure 1, which shows the worldwide relative search interest over time for the topics *data center* and *technology*.

From these data, it can be concluded that both topics have experienced substantial growth over the past five years. Although it may not seem obvious, there is a direct relationship between public interest and the motivation to learn.

Public interest in a particular topic or product may be the result of marketing efforts. When applied to a game and combined with an engaging context, it can increase the player’s level of interest even before gameplay begins [8].

Furthermore, the technological domain in which the project is situated is itself a motivating factor for learning. Young people aspiring to careers in technology, such as game development, are often attracted to computer science programs because they revolve around an "object of pleasure" (the game) [13].

Therefore, the developed project, *Port 0*, has strong learning potential, both direct and indirect, since it is situated within a field experiencing increasing hype. The attractiveness of an educational game is closely related to its content and to the connection that learners establish with its subject matter [10].

3 Design and development of Port 0 game

3.1 Design of Port 0 game

The game concept can be described as follows: a fantasy setting with science-fiction elements in which the player takes on the role of an engineer working in an

environment inspired by a data centre. The player's primary responsibility is to perform technical tasks. The main tasks are to buy and configure the hardware used in a data center, taking into account several aspects, such as for example, the processing capacity, the energy aspects, etc. Other tasks are presented in the form of word-based minigames, in which the player must write a set of commands with a specific purpose. In addition, the player must defend the data centre from attacks using a cannon.

The game also features several gameplay loops, among which the task loop and the attack loop stand out. The task loop is the most common and longest-lasting loop. During this loop, the player receives requests from clients and must choose whether to accept or reject them. If a request is accepted, the player must complete a word-based minigame inspired by entering commands in a terminal, in which a set of commands must be written to complete the task. In the attack loop, on the other hand, the player must use a cannon to defend the data centre from hordes of enemies attempting to attack the facility.

With this in mind, several game design techniques were applied to achieve the intended concept as effectively as possible, including:

- MDA Framework [18];
- Flow [19];
- Hook Model [20].

The MDA Framework [18] guided Port 0s design by directly embedding its theoretical pillars into the gameplay. Mechanics encompass typing Linux commands in WordRush, configuring server hardware, and operating the defense cannon. These spark Dynamics focused on resource management, balancing task revenue against energy expenses, and adapting to cyberattack interruptions. Ultimately, the Aesthetics deliver a sense of technical mastery and engagement by breaking task monotony with action-packed combat.

Flow [19] was one of the most important techniques, as it provides a framework for measuring and designing an experience capable of keeping the player in a state of flow. This state represents a balance between the player's level of challenge and their abilities, preventing them from becoming either bored or anxious.

The attack loop was specifically designed to help maintain the player in a state of flow. This type of gameplay loop breaks the potentially monotonous task-based gameplay and provides additional stimulation, helping to move the player away from a state of boredom.

Finally, the Hook Model [20] provides a theoretical and practical framework designed to encourage the development of habits through continuous interaction with a product. Furthermore, the model helps define behavioural patterns through which continuous interactions with the product can be encouraged, including in the context of the developed game. For example, visual appeal, appeal/curiosity for the theme and earning money resulting from tasks performed that are paid periodically.

3.2 Development of Port 0 game

Port 0 was developed using a set of tools and technologies, including *Unity* as the game engine, *C#* as the programming language, and *Blender* as the software used for the creation and texturing of 3D models.

The game is composed of several interconnected systems that work together to create a complete and interactive experience. The shop system allows the player to purchase a server. However, to do so, the player must go through a series of steps in which several components have to be configured. The purpose of this system is to introduce the player to the different components that make up a server, while also providing a brief description of each component.

Alongside the shop system, an economy system was developed to manage the transactions performed by the player, such as purchases and sales. It also manages periodic transactions, including recurring payments from clients for provided services and expenses such as energy costs.

The task system is responsible for randomly selecting tasks and presenting them to the player. This system enables the player to earn money both actively, by completing new tasks, and passively, through payments generated by previously completed tasks. This system constitutes a continuous gameplay loop that operates throughout the entire gameplay experience. The *WordRush* minigame, in which the player writes a set of Linux commands, is a direct consequence of this system, as completing the minigame is what concludes an accepted task.

Finally, the combat system is responsible for managing the hacker attacks that occur throughout the game. It operates periodically, with the primary purpose of breaking the task-based gameplay loop. It acts as a disruptive element that interrupts gameplay that could otherwise become monotonous, introducing greater variety, action, and entertainment to the player's experience.

4 Results

4.1 Evaluation Methodology

To evaluate the developed project, two questionnaires were applied — one before and another after the game session — aimed at measuring the participants' increase in knowledge. In this session, the player's goal was to reach the 1st DDoS attack. Simultaneously, the game sessions were observed to identify usage patterns and opportunities for UX improvement, analyzed according to Nielsen's usability heuristics [21], chosen because they were specifically developed to evaluate user experience. Telemetry data was also automatically collected through the Firebase platform, allowing the identification of anomalous interaction times at different moments of gameplay.

Problem	Severity	Heuristic	Solution
Lack of <i>feedback</i> in sound system/notifications/CPU config.	Low	Visibility of system status	Addition of an informative <i>label</i>
Cannon controls	High	User control and freedom / Consistency and standards	Invert commands
Player does not remember how to open the shop	High	Recognition rather than recall	Addition of a visual indicator on the UI
Player lacks awareness of the purpose of <i>hardware</i> components	High	Match between system and the real world	Addition of a label with a short component description in the shop
Lack of <i>feedback</i> in emails	High	Visibility of system status	Addition of a visual indicator that appears whenever there is an unread email
Camera too close to the player	Low	User control and freedom	Addition of camera zoom control

Table 1. Analysis of problems using Nielsen’s heuristics

4.2 Sample Characterization

The sample consists of 25 individuals of various genders and ages ranging from under 18 to over 30 years old, mostly concentrated between 18 and 29 years old (21 out of 25 participants). Regarding prior knowledge in computer science, 52% of the sample has an academic background in the area, while the remaining 48% have only user-level knowledge or none at all. This factor is relevant, as participants with prior training tend to show a less pronounced knowledge gain.

Age range	Individuals
Under 18 years old	2
18 to 24 years old	10
25 to 29 years old	11
30 to 34 years old	2
Over 35 years old	0

Table 2. Age range

Prior knowledge	Individuals
Academic degree in the area	13
User level only	4
None	7

Table 3. Prior knowledge in computer science

4.3 Knowledge Conveyed by the Game

The comparison between pre- and post-game responses reveals consistent knowledge gains in areas directly addressed by the game, namely hardware components (CPU and hard drives), architectures (highlighting ARM, whose recommendation appears in the shop associated with energy savings), CPU brands (IBM),

operating systems (Linux), web frameworks, and software installation methods. Table 4.3 summarizes the most significant variations.

In contrast, Ransomware recognition remained unchanged (18 responses before and after), which is consistent with the fact that this attack is not directly addressed in the game — unlike DDoS, whose recognition increased by 41.2% (from 17 to 24 correct responses). This suggests that knowledge gains are concentrated on content effectively taught through the gameplay experience, rather than a generic and non-specific improvement.

Evaluated knowledge indicator	Variation (before → after)
Recognition of IBM as a CPU brand	8% → 48% (+40 p.p.)
Recognition of Linux as an operating system	76% → 88% (+12 p.p.)
Recognition of the ARM architecture	56% → 84% (+28 p.p.)
Recognition of frameworks (e.g., Apache)	48% → 72% (+24 p.p.)
Software installation via terminal (correct answer)	15 → 24 (+60%)
Recognition of DDoS attack	17 → 24 (+41.2%)

4.4 Analytical Data Analysis (Telemetry)

Telemetry data collected via Firebase allowed the calculation of descriptive statistics (mean, standard deviation, and coefficient of variation, CV) for the main player actions:

Analytical metric (time)	Mean Standard deviation CV
Purchase of 1st server	125.94 s 84.88 s 67.39%
Attack duration (1st DDoS attack)	85.33 s 25.11 s 29.42%
Total time until the end of the 1st attack	85.33 s 25.11 s 29.42%

The purchase of the first server shows the highest variability (CV of 67.39%), possibly associated with the heterogeneity of the sample’s prior computer science experience, resulting in participants with formal education in the area executing this task faster, although the data does not allow establishing a direct causal relationship.

On the other hand, the attack duration (combat minigame) presents moderate variability (CV of 29.42%), lower than that observed in other actions. This result is attributed to the straightforward and familiar nature of this mechanic, similar to mechanics found in other video games. The total time until the completion of the 1st attack (CV of 35.90%) reveals, through the average trendline, two distinct performance groups, reflecting the diversity of the sample. Finally, the WordRush word minigame presented average times between 30 seconds and 1m30s per category, an interval considered positive for avoiding predictability without making tasks overly long.

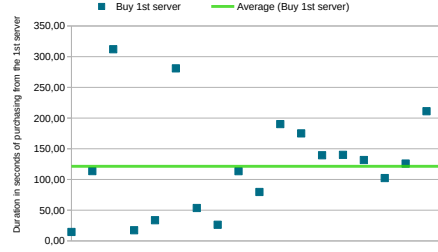


Fig. 2. Time required for players to purchase the first server (high dispersion relative to the mean).

4.5 Summary

Overall, the results in Chapter 5 suggest that the game contributes to a measurable increase in technical knowledge specifically related to the content covered (hardware, architectures, operating systems, frameworks, and DDoS attacks), with no equivalent effect on unaddressed topics (Ransomware). The usability analysis (Nielsen’s heuristics) identified concrete issues, mostly regarding visual feedback and feature discoverability, which were resolved with targeted solutions. Telemetry data shows heterogeneous performance in initial tasks (server purchase) but greater consistency in core game mechanics (attack), suggesting effective learning of the primary mechanics throughout the session.

5 Conclusion

This paper presented an evaluation of **Port 0**, a resource management simulation game designed to foster both direct and indirect learning within data center management and foundational technology concepts. By integrating mechanics such as server hardware configuration, command-line minigames, and dynamic DDoS defense loops, the game aims to provide an engaging educational experience without sacrificing commercial entertainment value.

The empirical evaluation, conducted with a sample of 25 participants across varied technical backgrounds, demonstrates that interactive simulation mechanics can lead to measurable knowledge acquisition. Post-game assessments revealed notable improvements in technical areas explicitly embedded in gameplay—such as server architecture awareness, operating system commands, software deployment, and cyberattack mitigation—while domain concepts omitted from the game mechanics showed no change. Furthermore, telemetry data and usability evaluations based on Nielsen’s [21] heuristics proved essential for identifying user experience bottlenecks, enabling iterative interface and feedback improvements.

Overall, the findings suggest that contextualized, mechanics-driven video games serve as effective pedagogical tools for technical education. Future work

will focus on expanding the scope of operational challenges within **Port 0**, introducing advanced security scenarios such as ransomware mitigation, and conducting longitudinal studies with larger cohort groups to assess long-term knowledge retention.

References

1. Author, F.: Article title. *Journal* **2**(5), 99–110 (2016)
2. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) *CONFERENCE 2016, LNCS*, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). <https://doi.org/10.10007/1234567890>
3. Author, F., Author, S., Author, T.: Book title. 2nd edn. Publisher, Location (1999)
4. Author, A.-B.: Contribution title. In: 9th International Proceedings on Proceedings, pp. 1–2. Publisher, Location (2010)
5. LNCS Homepage, <http://www.springer.com/lncs>, last accessed 2023/10/25
6. Hirschfeld, T.: *How to Master Video Games*, Bantam Books, New York (1981)
7. Miller, M.: *Video Games and Indirect Learning*, Advances in Game-Based Learning (2021)
8. Martens, A., Diener, H., Malo, S.: *Game-Based Learning with Computers Learning, Simulations, and Games*, vol. 5080, Berlin, Heidelberg (2008)
9. Villareale, J., Biemer, C. F., Seif El-Nasr, M., & Zhu, J. *Reflection in Game-Based Learning: A Survey of Programming Games*. In *Proceedings of the International Conference on the Foundations of Digital Games (FDG '20)*. ACM, 2020. doi:10.1145/3402942.3403011
10. Marques, B. P., Reis, R., Cardoso, M.: *Games: The Motivation in Engineering Education*. In: Ninth International Conference on Technological Ecosystems for Enhancing Multiculturality (TEEM'21), pp. 395–399. ACM, New York (2021). doi:10.1145/3486011.3486483
11. Sari, N. M., Yaniawati, P., Supianti, I. I., Indriani, R.: *Digital Game-Based Learning Interventions on Students' Numeracy Skills and Engagement*. *Formatif: Jurnal Ilmiah Pendidikan MIPA*, 15(1), 39–50 (2025). doi:10.30998/formatif.v15i1.23356
12. Ibrahim, R., Jaafar, A.: *Using Educational Games in Learning Introductory Programming: A Pilot Study on Students' Perceptions*. In: IEEE International Conference, pp. 1–5. IEEE (2010). doi:10.1109/ITSIM.2010.5561324
13. Baxter-Webb, J.: *The Role of Gaming Platforms in Young Males' Trajectories of Technical Expertise*. *Transactions of the Digital Games Research Association (ToDiGRA)*, 2(3), 89–115 (2016). doi:10.26503/todigra.v2i3.58
14. Lindberg, R. S. N., Laine, T. H., Haaranen, L.: *Gamifying Programming Education in K-12: A Review of Programming Curricula in Seven Countries and Programming Games*. *British Journal of Educational Technology*, 50(4), 1979–1995 (2019). doi:10.1111/bjet.12685
15. Hello Games: *No Man's Sky*. Video game. Hello Games, Guildford (2016)
16. Nintendo EPD: *The Legend of Zelda: Breath of the Wild*. Video game. Nintendo, Kyoto (2017)
17. Cambridge Dictionary: *Hype*. <https://dictionary.cambridge.org/dictionary/english/hype>, last accessed 2026/08/03
18. Hunicke, R., LeBlanc, M., Zubek, R.: *MDA: A Formal Approach to Game Design and Game Research*. In: *Proceedings of the AAAI Workshop on Challenges in Game AI*, vol. 4, no. 1, p. 1722. San Jose, CA (2004)

19. Chen, J.: *Flow in Games (and Everything Else)*. Communications of the ACM, 50(4), 31–34 (2007)
20. Eyal, N.: *Hooked: How to Build Habit-Forming Products*. Penguin UK (2014)
21. Nielsen, J.: *Enhancing the Explanatory Power of Usability Heuristics*. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, pp. 152–158 (1994).